Project Title

Learning Adaptive Network Routing with Reinforcement Learning and Graph Neural Networks

Student Name: Nuo Chen

Objective

The objective of this project is to design and evaluate a reinforcement learning (RL)-based network routing algorithm that leverages Graph Neural Networks (GNNs) to learn adaptive routing policies in dynamic network environments.

Specifically, the project aims to:

- Develop an RL agent that selects routing decisions (next-hop or path) in a network graph
- Use a GNN-based model to encode network topology and traffic conditions
- Optimize routing performance metrics such as latency, throughput, and congestion

Significance

Traditional network routing protocols (e.g., shortest path routing) rely on static heuristics and cannot adapt well to dynamic traffic patterns and changing network conditions. As discussed in recent research, routing optimization is a complex sequential decision-making problem with large state and action spaces, making it a natural fit for reinforcement learning

Recent advances in deep reinforcement learning and graph neural networks provide new opportunities to model complex network structures and learn generalized routing strategies. This project is important because:

- It explores the integration of RL and GNN for system-level optimization
- It addresses limitations of heuristic-based routing in dynamic environments
- It contributes to ongoing research on learning-based networking systems

Methodology

The project will be implemented through simulation and consists of the following components:

1. Environment Setup

- Model the network as a graph $G = (V, E)$, where nodes represent routers and edges represent

links

- Generate synthetic network topologies (e.g., random graphs, grid, scale-free networks)
- Simulate traffic flows between source-destination pairs

2. State Representation

- Node-level features: queue length, load, link utilization
- Edge-level features: bandwidth, latency
- Use a Graph Neural Network (GNN) to encode the global network state

3. RL Agent Design

- Action: select next-hop node (or path) for each packet/flow
- Reward:
    - Negative latency
    - Penalty for congestion or packet loss
- Use a DRL algorithm such as:
    - Deep Q-Network (DQN), or
    - Proximal Policy Optimization (PPO)

4. Training and Evaluation

- Train the agent in dynamic traffic scenarios
- Compare against baseline routing methods:
    - Shortest Path Routing
    - Random Routing

5. Extensions

- Multi-agent RL (each node as an agent)
- Generalization to unseen network topologies
- Comparison with non-GNN models

Delivery (what you expect to provide in the final report)

The final deliverables will include:

- Implementation code (Python + PyTorch)
- Simulation environment for network routing
- Experimental results and visualizations:
    - Average latency

    - Throughput
    - Congestion metrics
- Comparative analysis with baseline methods
- A written report discussing methodology, results, and limitations